

S P E C T R A V I D E O

U S E R S G R O U P -- Wgtn

Volume 3, Number 2 -- September 1986

The next meeting is on Monday, September 15th at 7.30 in the Staff Common Room, Wellington Clinical School, Main Street Newtown. The following meeting will be on the 20th of October 1986. (ie as usual the 3rd Monday of the month.)

NOTE - VENUE FOR NOVEMBER MEETING MAY NOT BE THE CLINICAL SCHOOL
If you don't hear at this Meeting contact a committee member.
NOTE ALSO THAT THE NEXT NEWSLETTER IS DUE APPROX. 7 NOVEMBER 1986

Wellington SV Users Group

Committee Members

Chairman ----- Darryl Alison Ph 85522 (Paraparaumu)

Secretary ----- Joseph Dol 889275

Treasurer ----- Ross Fletcher 788454

Don Stanley 896379

Garry Clark 367872

Newsletter Committee

Steve Hanson 843495

John Matthews 327805

Other specific areas of responsibility for committee members are
Don -- Software Consultant
Garry -- Radio Access & public domain tape software;
John -- Library;
Darryl -- Program evaluation committee chairman & Kapiti users representative.

C L A S S I F I E D S

SITUATIONS VACANT

WANTED - ONE HARD-WORKING CO-EDITOR FOR THIS ESTEEMED PUBLICATION
- HOURS ARE NOT TOO UNREASONABLE - AND IT'S A GREAT WAY TO LEARN
A LOT ABOUT THE CLUB. Situation only available because incumbent
(JOHN) has been lured away by competition (- 68000 processor -
windows - mouse - the works ! - what can we do??).

Contact STEVE or anyone on the Committee !

GAMES SCORES

Spectron	12910	Robert Campin
Sector Alpha	2445200	Garry Clark
Kung Fu	82435	Garry Clark
	48705	Garry Clark (*)
S C Force	748600	Deirdre Chin
Telebunny	113520	Garry Clark
	32720	Garry Clark (+)
	31670	Kalima Collinson (*)
Omac	35360	Richard Bremford
	25700	Don Stanley (+)
Ninja	251350	Nicola Tuinman
Tetra	284450	Julian Stone
Turboat	68390	Garth Thornton (*)
Turboat	104610	Taane Clark
Sasa	172950	Steve Lacey
Frantic Fred	21720	Rachel Hughes (level4)
Flipper Slip	37640	Garth Thornton
Bats Alive	3540	Colin Chin (*)
Galaxy	1447	"
Jumping Jack	2600	"
TV Nightmare	7040	"
	3640	" (*)
	4980	Deirdre Chin (+)

(+) = started in normal level

(*) - started in arcade level

otherwise practice level or unknown.

Although it is nice not to have to update the scores all the time, it would be nice to see a few more changes in the scores above. C'MON all you gamesmasters - show us how good your skills are !!

Club Sponsor

Action Computers -- P.O Box 7442, Wellington South.

Photocopying from Library

Anybody requiring articles from the library to be photocopied please send request to the LIBRARIAN, not any other committee member. We have had problems with articles not being copied when requested, please ensure that requests go to the librarian.

Also membership renewals should be sent to Box 26050, Newlands, Wellington, not to any other box, otherwise it is possible that the mailing list will not get updated.

A note to would-be contributors

Don't let *Anyone* or *Anything* put you off. The newsletter is made from your contributions, if we don't receive any, we can only produce a poor magazine.

Presently we are lacking in BASIC programs and in hints to beginners, and from beginners. We try to cater to all club members, but lately our we have been receiving mainly specialty items, which are of little benefit to many. Among the many things we would like to see arriving in the mail (addressed to *The Editors*) are:

- Programs - Basic, Pascal, Assembly, YouNameIt.
- Programming hints, and techniques.
- Utilities - any type, as long as they are self explanatory or come with instructions.
- News - anything you think other club members might like to know.
- Problems - we will find someone to answer them if we can't.

Everything we receive will be considered.

Public Domain CP/M

Anyone wanting anything from the library of CP/M PD software, please write to the Librarian at the club address, specifying either which disc, or which programs you would like. As my drives are single sided, a whole disc from the list now occupies up to 2 full discs. Most of the programs on the discs are 'Squeezed', this means they have been run through a function in New Sweep (NSW, Nsweep etc. but not the earlier Sweep) so as to take up as little room as possible. They all have a 'Q' as the 2nd letter in their extension, and can be easily unsqueezed by using New Sweep to tag them and then the 'Q' option to unsqueeze them, usually onto another disc. Please send discs, rather than asking us to purchase them for you, and as my drives are slightly "Non-Standard", you would be unable to read them if formatted on my drives, so please send them formatted. Please also include, either a stamped return mailer, or add 60c to your payment to cover postage. We will return your disk in your packaging.

C O R R E S P O N D E N C E

We have received letters from a couple of members, which I felt should be answered in the newsletter. These are:

* Joan Newlove has written with a couple of questions.

Firstly, the Public Domain:

Public Domain Basic Programs are available on Cassette through Garry - through the Club's P.O. Box. Five NEW tapes are available, and a list of their contents appears elsewhere in this issue. Previous tapes were listed in the December 85 Issue (Page 11). These Basic programs have been written to run on any SVI328 however many will run on an unexpanded 318 - Although these programs are on cassette, they include a large number of Disk BASIC utilities etc., and although some are designed to run on an 80 column screen - these are generally not too hard to convert.

Public Domain CP/M Programs are available through Ross Fletcher - also through the Club's P.O. Box. John has been through the Disks, and has outlined what is on them in the June issue. A full list of the programs on the P.D. Disks was published in the Jan/Feb 1986 Issue (Although please note that copies of these disks are now only available through the Club P.O.Box - not through Deane as stated in that article).

Secondly, Educational Programs in CP/M:

I'm afraid that Public Domain is very reliant upon the expertise of CP/M users/programmers to maintain a good variety of program titles. In my experience, there is a decided lack of 'Educational - Type' programs in any Public Domain Library. However you may be able to view the Public Domain Libraries of other CP/M Users Groups, ie Osborne/Kaypro

A word of warning, however - make sure that their version of CP/M is 'compatible' as CP/M 3.0 (as used by say 'Microbee users') is not 100 percent compatible with our CP/M 2.22 or CP/M 2.0. Also, any program using Sound or Graphic effects is generally incompatible from one machine to another - so we are reliant upon either the expertise of our own programmers, or on the availability of text-based programs from other Micro users.

So here lies a challenge and a request - is there anyone out there with access to (or a flair for writing) educational programs under CP/M? - say school software, (Maori) language tuition programs etc?

NOTE There are quite a few utilities, word processing, and Database programs in the Public Domain of very good standard - but if these fall short of your requirements - for anything else, you have to start looking in the Commercial area.

* Jim Megenis has recently delved into the Public Domain but from this comes a few Questions:

Firstly, the 'VDO' text editor is written for a Disk-based system with an 80 Column Card - where can one be got?

I too have suffered from this problem - and I'm in fact writing this newsletter on Wordstar - using a borrowed 80 column card. - I do, however have one on order through Don, and talking with him today, he has told me that if there are another couple of requests (ie via ACTION COMPUTERS - his ad's in the front of the Newsletter) he will "search the world" for a new supplier. The last lot are coming from Australia and there is an indication that there may be some available through other countries as well - so if you see or hear of any (overseas - or otherwise) hardware suppliers, it may be of some assistance to your fellow club members to let Don know.

Secondly - in the meantime does anyone know how to patch / hack / update VDO or any similar word processor/editor to run on 40 Column screen under CP/M?

As far as I know, this can only be achieved by writing a completely new set of 'Drivers' to replace/supplement the 80 Column Card Drivers within the program itself so that the program can access the SVI's Video Display Processor. This would also involve redesigning the screen format, - and writing some pretty fancy scrolling routines if the text editor is to show a full page-width of characters (65 characters for a normal A4 page). It would probably be better to write a 40 Column Editor from scratch. How's that for a challenge !

Thirdly - disassemblers under CP/M.

The CP/M distribution disk contains DDT - the 'Dynamic Debugging Tool' which is an assembler/disassembler which can be used to disassemble any CP/M program into '8080 Assembler Mnemonics. There is also available a Z80 CP/M disassembler called BEEMON - which is readily available through dealers (Eh ! Don).

Finally - Bob Curwood's SWITCH routine V1.3 as listed in our June issue is designed to use the 'shadow' RAM area used by CP/M but normally unavailable to BASIC programs. It merely copies a BASIC program from one bank to the other, and returns you to the original Bank. You must therefore have an unexpanded 328 or expanded 318 so that the shadow bank exists.

+ + + + +

FILE UTILITY

The following program was written for those people with disc drives but without 80 column cards or a disc saving word processor. During its development I have learnt a lot about files which was valuable but time consuming. It is presented here to save you the same trouble.

One small catch with it is the inability to recognise cassette file names in the tape to disc mode. I would advise that return or enter is typed in response to the request for an existing cassette file name. The programme then runs normally.

'File' is useful when reading or tranfering to disc unknown cassette files. The well known U.S Presidents program is a good one to work up to.

Remember, all files or programmes must be saved in ASCII form by using the SAVE "1:file",A command for disc files or SAVE "file",A for cassette files.

If you dont do this the program may or may not run normally but you will end up with a mess on the screen.

JIM OLIVER

```
10 REM ** File Utility Program **
20 REM J.A.Oliver SVUG (Wgtn) July '86
30 REM Thanks to D.S & G.D
40 '
50 '
60 ' ***** WARNING !!!!!
70 ' -----
80 ' IF ANYTHING GOES WRONG,CTRL STOP
90 ' THEN CLOSE:FILES BEFORE CONTINUING
100 '
110 '
120 '
130 NF=0:DN$="":CN$="":MAXFILES=4:CLEAR1200
140 ONSTOPGOSUB1280:STOPON
150 ON ERROR GOTO 1620
160 SCREEN0,0:CLS:COLOR 1,10:IV$=CHR$(27)+"p":NV$=CHR$(27)+"q"
    :I=0
170 DATA Tape file to disc.
180 DATA Disc file to tape.
190 DATA Append files.
200 DATA View a file.
210 DATA Print files from disc.
220 DATA list disc files.
230 DATA music ?!!
```

```

240 LOCATE7 ,0:PRINTIV$" FILE UTILITY MENU "NV$
250 PRINT:PRINTTAB(4)IV$" FILES MUST BE IN ASCII FORM "NV$
260 NP=7:REM No of programs
270 FOR I=1 TO NP:READ P$
280 LOCATE 6, (2*I+3),0:PRINTI;" ";P$
290 NEXT I
300 LOCATE0,20:PRINTIV$" Press Program No to continue "NV$;
310 LOCATE0,22:PRINTIV$" or 0 to quit "NV$:LOCATE20,22:INPUT P
320 IF P>7 THEN 310
330 IF P=0 THEN 1280
340 CLS
350 ON P GOTO 400,560,730,1050,1190,1520,1540
360 '
370 '
380 '
390 REM tape to disc.
400 GOSUB 1290
410 OPEN"cas:"+CN$ FOR INPUT AS #1
420 OPEN"1:"+DN$ FOR OUTPUT AS #2
430 IF EOF(1) THEN GOTO 480
440 LINE INPUT#1,A$
450 PRINT #2,A$
460 A$="":LOCATE15,10:PRINT"X fingers!"
470 GOTO 430
480 CLOSE #1:CLOSE #2
490 PRINT:PRINTPRINTTAB(15)"All done":PRINT:PRINT
      :PRINTTAB(9)"For Menu strike a key"
500 RE$=INPUT$(1):GOTO 130
510 '
520 '
530 '
540 '
550 REM disc files to tape
560 GOSUB 1290
570 OPEN"1:"+DN$ FOR INPUT AS #1
580 OPEN"cas:"+CN$ FOR OUTPUT AS #2
590 IF EOF(1) THEN GOTO 650
600 LINE INPUT#1,A$
610 PRINT #2,A$
620 LOCATE15,10:PRINT"X fingers"
630 A$=""
640 GOTO 590
650 CLOSE #1:CLOSE #2
660 PRINTTAB(15)"All done":PRINT:PRINT
      :PRINTTAB(9)"For Menu strike a key":RE$=INPUT$(1)
      :GOTO 130
670 '
680 '
690 '

```

```

700 '
710 REM To APPEND disc files;file 2,
720 REM file 3 and file 4 to filename
730 GOTO 1330
740 OPEN "1:"+DN$ FOR APPEND AS #1
750 OPEN"1:"+DN$(3) FOR INPUT AS #4
760 OPEN"1:"+DN$(2) FOR INPUT AS #3
770 OPEN"1:"+DN$(1) FOR INPUT AS #2
780 OPEN"1:"+DN$   FOR APPEND AS #1
790 IF EOF(2) THEN GOTO 840
800 LINE INPUT #2,A$
810 PRINT #1,A$
820 A$="":LOCATE14,10:PRINT"Running O.K"
830 GOTO 790
840 CLOSE 2
850 IF NF=1 THEN 990
860 IF EOF(3) THEN GOTO 910
870 LINE INPUT #3,A$
880 PRINT #1,A$
890 A$="":LOCATE14,10:PRINT"Still O.K!! "
900 GOTO 860
910 CLOSE 3
920 IF NF=2 THEN 990
930 IF EOF(4) THEN GOTO 980
940 LINE INPUT #4,A$
950 PRINT #1,A$
960 A$="":LOCATE8,10:PRINT"Whew! getting there"
970 GOTO 930
980 CLOSE
990 CLOSE:PRINTTAB(15)"All done":PRINT:PRINT
      :PRINTTAB(9)"For Menu strike a key":RE$=INPUT$(1)
      :GOTO 130

1000 '
1010 '
1020 '
1030 '
1040 REM To view a file
1050 GOSUB 1420
1060 OPEN A$+N$ FOR INPUTAS#1
1070 IF EOF(1)THENGOTO1120
1080 LINE INPUT#1,A$
1090 PRINT A$
1100 A$=""
1110 GOTO1070
1120 CLOSE1
1130 PRINTTAB(15)"All done":PRINT:PRINT
      :PRINTTAB(9)"For menu strike a key":RE$=INPUT$(1)
      :GOTO 130

1140 '
1150 '
1160 '

```



```

1170 '
1180 REM copy JUST WRITE Jnr. files from disc to printer
1190 GOSUB 1470
1200 REM ****      insert printer escape codes here      ****
1210 OPEN"1:"+DN$ FOR INPUT AS #1
1220 IF EOF(1) THEN 1260
1230 LINE INPUT #1,A$
1240 LPRINT A$
1250 GOTO 1220
1260 CLOSE 1
1270 PRINTTAB(15)"All done":PRINT:PRINT
      :PRINTTAB(9)"For Menu strike a key":RE$=INPUT$(1)
      :GOTO 130
1280 CLOSE:SCREEN,1:LOCATE,,1:COLOR15,4,5:ON ERROR GOTO 0:END
1290 LINE INPUT "Cassette file name or <cr> : ";CN$
1300 PRINT:PRINT
1310 LINE INPUT "Please input disc file name: ";DN$
1320 CLS:RETURN
1330 INPUT"No. of files for APPENDING (max.3)";NF
1340 IF NF>3 THEN 1330
1350 PRINT:PRINT: LINE INPUT"File name to be APPENDED to: ";DN$
1360 PRINT:PRINT
1370 FOR A=1 TO NF
1380 PRINT"Input file number";A;" file name: ";
1390 LINE INPUT DN$(A)
1400 PRINT:NEXT A
1410 CLS: ON NF GOTO 770,760,750
1420 INPUT"(D)isc or (C)assette file : ";A$:PRINT:PRINT
1430 IF A$="D" OR A$="d" THEN A$="1:"
1440 IF A$="C" OR A$="c" THEN A$="cas:"
1450 LINE INPUT"File name to display ? ";N$
1460 CLS:RETURN
1470 PRINT:PRINT:PRINT
      :PRINTTAB(6)"It is recommended that you use"
1480 PRINT: PRINTTAB(6)"the original editor to print"
1490 PRINT: PRINTTAB(10)"out your final copy.
1500 PRINT:PRINT: LINE INPUT"File name to print please: ";DN$
1510 CLS:RETURN
1520 COLOR1,3:CLOSE:FILES:PRINT
      :PRINTTAB(6)"Room for";DSKF(1);"file entrys"
1530 PRINT:PRINT:PRINTTAB(6)IV$" For Menu strike a key "NV$
      :RE$=INPUT$(1):GOTO 130
1540 REM *****echo notes*****
1550 COLOR7,7
1560 PLAY"t180l16o2","t200l32o2","t225l64o2"
1570 B$="cdefgab03cdefgab04cdefgab05cdefgab06cdefgab"
1580 PLAY"xb$";,"r4xb$";,"r8xb$";"
1590 FOR T=1 TO 1300:NEXT T
1600 CLS:COLOR 15,4:LOCATE12,8:PRINT"Who said music ?"
1610 CLOSE:PRINT:PRINT"For the Menu (ie sensible) strike a key"
      :RE$=INPUT$(1):GOTO 130

```

```

1620 IF ERR=53 THEN RESUME 1670
1630 IF ERR=54 THEN RESUME 1680
1640 IF ERR=19 THEN RESUME 1690
1650 ON ERROR GOTO 0
1660 GOTO 1280
1670 CLS:LOCATE10,10:PRINTIV$" CHECK FILE NAME "NV$
      :FOR T=1 TO 1500:NEXT T:GOTO 130
1680 CLS:LOCATE10,10:PRINTIV$" CLOSING FILES "NV$
      :FOR T=1 TO 1500:NEXT T:CLOSE:GOTO 130
1690 CLS:LOCATE5,10:PRINTIV$" DEVICE INPUT/OUTPUT ERROR "NV$
      :FOR T=1 TO 1500:NEXT T:GOTO 130

```

+++++

+++++

EXPLORING BASIC - PART 2

BY L. A. Dunning

BASIC is no more than a machine code program. This rather self-evident point is normally not apparent to beginners with microcomputers. As such, it can be divided into several different areas, each with certain functions. A typical BASIC interpreter might be composed of the following modules:

- System Boot
- Interpreter
- Editor
- Statement Code
- Functions
- Input/Output Drivers
- Keyboard Drivers
- Vectors
- Buffers
- Tables

Let's take these one at a time. The System Boot is the section of code used when the system is initially turned on or reset. This will set up the system in an appropriate manner depending upon what hardware configuration you have. A Disk system will expect some program on disc to initialise, whereas a tape system will not bother with this; thus the SV's can be configured for Tape, CP/M or their own Disk Operating System (DOS). The Interpreter is more or less the "real" BASIC program. When active, it checks input via the keyboard or other sources, decides if it is important and if it is not either ignores it or generates an error message. It "drives" the BASIC and is partly why interpreted (as opposed to compiled) BASIC appears so slow.

The Editor is necessary if the user is to modify or to enter any sort of BASIC program. This is really a subsection of the Interpreter but changes the Subject Code. Each Statement Code is merely the Object Code for a particular Statement, after that Statement has been read, syntax checked and variables assigned to proper buffers or locations. This is the "meat" of BASIC, whereas the Interpreter is the "bones", so to speak. The Functions are sections of code used by more than one statement module, - used to convert numbers, check status, et cetera.

Each computer needs to relate to external devices in order to communicate with the user or provide hard copy of information. Such devices include the Keyboard, Screen, tape drive, Disk drive, Printers et cetera. In order to do so, it needs a hardware configuration backed by software programs called "drivers". Drivers are normally written with particular devices in mind but a good system will be constructed so that the users can write their own drivers. I've listed the Keyboard Driver separately since it is important for most input from the user and is the most basic peripheral for any computer.

Each computer needs a certain amount of RAM to hold variables, pointers et cetera; these I have divided into three types: Vectors, Buffers, and Tables. A vector is a location in RAM that the program will jump to and be further directed to another location. These can be manipulated so that program flow can be diverted other than to the normal point of flow. (See Don's examples in last issue, or in Bits & Bytes: WSVUG Ed). Used carefully, they can add extra basic commands, perform checks or alter information flow. There is a series of vectors in high memory on the SV's, mostly used for Disk Basic.

Buffers are sections of memory used to hold temporary values for processing. The most common uses of Buffers are string manipulation, file input/output and screen handling. There are several buffers in high memory, (many of which I have yet to determine the exact purpose of) used for all of these. A Table is a more regular area for saving values and variables. Several Tables exist both in RAM and ROM and basic maintains Tables to indicate the type and location of variables, the variables themselves, and arrays. Apart from the stack, these normally produce the biggest overhead in many programs.

To work in any capacity using Machine code or assembly language on a computer, the programmer should have a working knowledge of the input/output Drivers, or at least the hardware connections involved, otherwise how does one manipulate the thing? Next article, I will talk about the Video Display and how it's interfaced with the SV and also of a few tricks you may not know.

You may remember listing 2 that was printed last month and the strange results when line 160 was listed after being altered. This is because the code used to list the program line is different from that used to edit the line. The Editor normally eliminates bytes in the line less than 32 except certain functions, literals, and end of line byte. The lister doesn't expect to find such characters except a part of these and so produces uncertain results. What are the bytes that do this? - A list is given below:

Byte	Use
9	Produces a TAB when listed
10	Produces a linefeed, unknown results
11	Prefixes an Integer value as represented by '&O'
12	Prefixes an Integer value as represented by '&H'
13	Line number reference prefix '*'
14	Prefixes an Integer value : two bytes follow
15	Prefixes a single byte value: One byte follows
17--27	Equals an absolute value of byte - 17
28	Prefixes an integer value : Two bytes follow #
29	Prefixes single precision: Four bytes follow #
30	Line number reference prefix # *
31	Prefix double precision: Eight bytes follow #
*	The next two numbers indicate relative position of line in relation to current line.
#	Editor uses these bytes in preference to others.

The interpreter uses the prefixes to determine what to expect next, if they are used as cursor control characters in literal strings, the video driver is in control and no problems results. When edited however, the line is first listed and the editor takes the result as verbatim and so the value are changed. Notice there is no "&B" prefix, this is because it is represented by ASCII as 26/42 and "1" & "0" in the value as 31/30.

A few points aside from the above discussion; What is the least memory using method of reading data statements for sprites and machine codes? In each case each bit of data is a single byte. If these are read individually, they eat up one byte per piece of data for commas and look tacky. Listing "1" demonstrates a method that uses an optimum number of bytes. It's based on the CALL CHAR statement from the TI99/4A.

One of the annoying things about the user's manual is its general inaccuracy. Error messages for WHILE and WEND statements are listed, but these statements are not implemented. WHILE/WEND works as follows: The Interpreter comes to the WHILE statement which also has a condition attached to it, if this condition is true, the logic falls through to the next statement, it then continues on until it reaches the WEND statement and loops back to the WHILE statement; if the condition is false it skips to the next statement after the WEND.

sets. One set was identified with both hydrogen and

Listing 1

```
130 DATA 003C7FDFF3F1212,1030F1323E1E236,FFFFFFFFFFFFFFFF
```

Listing 2

100 NEXT

+ + + + + + + + + + + + + + + + + + + +

CLASSIFIED S

FOR SALE

CENTRONICS Printer Interface - Cost - \$115 - (Negotiable)

Ring Chris McIntyre - 759393 (Home)

737777 (Work - leave a message)

A bargain - even if you don't have a printer - you will find you need one soon enough.

BOOK REVIEW

"EXPLORING ARTIFICIAL INTELLIGENCE ON YOUR MICROCOMPUTER"

by Tim Hartnell

Interface Publications, London and Melbourne, 1984

The programs listed in this book were written in a subset of BASIC which is compatible with both Microsoft BASIC and MSX BASIC, and hence is almost compatible with Spectravideo BASIC. The only modification which is necessary for the Spectravideo is to change the RANDOMIZE (VAL(RIGHT\$(TIME\$,2))) to X=RND(-TIME). Note that TIME is an integer not a string.

This is a very interesting book, not too heavy, in fact some may find it doesn't go far enough.

There is a program which plays TIC-TAC-TOE and learns from experience as it plays. SYLLOGY is a program which makes logical inferences on the information that you feed it, and gives the impression that it can reason. SNICKERS is a simplified draughts game using tree searching, mini-maxing, evaluation functions and an alpha-beta algorithm. This is perhaps the most sophisticated program in the book, and yet it is explained in such a way that most of us could understand it.

BLOCKWORLD is a simplified version of a well-known AI program called SHRDLU, in which you tell the computer to shift blocks in relation to each other. It involves parsing of natural language, although this topic is not covered in anywhere near the depth that I would have liked. Fortunately there is a good bibliography at the back of the book.

DOCTOR is the most complete version of ELIZA ever published in BASIC, according to the author. It carries out a conversation with the user, as if it were a psychiatrist. You can of course modify any of the programs, and this one in particular can be customised to your own taste. It is also the longest in the book - about 300 lines.

TRANSLATE is a very simple program for translating English into French (fairly appalling French actually). With the addition of more vocabulary it would be more useful, but it still wouldn't know the first thing about French grammar. It will need a lot of jazzing up before it will be much use for doing your homework, I'm afraid.

The last two chapters in the book present a gentle introduction to expert systems and self-learning programs and there are six example programs for the reader to try.

In addition to an excellent bibliography there is an appendix on improving your programming technique, and a glossary of AI terms, both of which may be useful to some readers. I would recommend this book as a simple introduction into artificial intelligence for those who are interested.

Donald Rogers

DISK GAMES

(A small utility which enables you to keep a backup copy of your game cassettes on disk - for ease of loading too! :ED)

Recently I read a request from someone wanting to know how to save SVI cassette programmes such as Armoured Assault or Spectron to Disk.

These games consist of two separate programs, the first loads the second which is a screen save.

I saved these to disk using the following method:

(1) CLOAD the first small BASIC program, (loadg). Leave the Data Cassette as is - don't turn off the play button.

(2) Change line 10 from CLOAD"GAME" to LOAD "1:game" then SAVE the program to disk as "name" in drive one.

(3) NEW anything in memory

(4) Now write the following;

```
10 SCREEN 1
20 CLOAD "GAME"
30 SAVE"1:game",S
40 SCREEN 1
50 PRINT"ALL FINISHED"
60 GOTO 60
```

(5) RUN the program which you have just entered. This will LOAD the screen into VRAM and then SAVE this screen to disk.

(6) When the screen clears and prints the message, turn off the Data Cassette, do a ^STOP to gain control of the screen and then check your disk files.

(7) You should now have two files on disk: "name" and "game". The "game" file should have a # after it indicating a screen saved file. This will take up 4 tracks.

Now you can load and run the "name" file which will in turn load the VRAM quite a bit faster than tape!

You should now get the SVI logo and the game instructions, - it worked for me!

The "game" above can be any name but must be unique so your system can tell which is the correct screen file to load with the correct game.

I used "AAss" and "game1", and "spec" and "game2" for "name" and "game" for Armoured Assault and Specmon respectively but you can use any name you like.

J.A. Oliver

TITBITS ... by Don Stanley

(1) Function Keys

Both CPM and BASIC (on SVI or MSX) treat the function keys as ending when a null (ascii 0) is encountered. However there is no reason why you shouldn't use characters after the null. These characters would be part of the function keys, but are NOT interpreted when the key is pressed.

This has many implications. One is for Wordstar users. You can redefine the keys to correspond to Wordstar instructions, then have a null, then have a meaningful description for the keys. For instance you could define F1 to be a DELETE TO END OF LINE, in other words, CNTL/Q followed by CNTL/Y (or just a Y). The next character in the key definition would be a null, then you are free to insert any text you like (ending with a null). Thus you could have the text DELETE EOL after the null.

Getting these characters in there can be accomplished in several ways. One is to write a small machine code program which creates a COM file. This COM file sets up the keys. Here is how you do this ...

```
org      100h
;
; fkey setup for Wordstar
;
keystr equ 0e39bh ; where Xpress CPM keys are stored
; keystr equ 0fa1eh ; where 328 cpm2.22 keys are stored
;
mvi      a,0edh    ; move an LDIR into program
lxi      h,cont
mov      m,a
inx      h
mvi      a,0b0h
mov      m,a
lxi      h,keydef   ; move the Wordstar keys
lxi      d,keystr
lxi      b,160
cont:    ds         2 ; replaced by LDIR when run
ret
keydef:  db 11h,59h,0,'DELETE EOL',0,0,0
         db 11h,43h,0,'END OF FILE',0,0,0
         db 11h,52h,0,'TOP OF FILE',0,0,0
         db 0bh,42h,0,'START BLOCK',0,0,0
         db 0bh,4bh,0,'END BLOCK',0,0,0,0
         db 0bh,56h,0,'MOVE BLOCK',0,0,0,0
         db 0bh,43h,0,'COPY BLOCK',0,0,0,0
         db 0bh,59h,0,'DELETE BLOCK',0,0,0,0
         db 0bh,53h,0,'SAVE & CONT',0,0,0,0
         db 0bh,44h,0,'SAVE - MENU',0,0,0,0
```

This method is quite useful, as you can define various keys

to suit the situation, then load the keys you require under different circumstances.

Having entered the program above (using the N option of WS or using ED and call it WSKEYS.ASM, or something.ASM) you save it, then run the program ASM by typing ASM WSKEYS. This will create a number of files. Enter the command LOAD WSKEYS which creates a file called WSKEYS.COM . You can now load the keys by entering WSKEYS.

The method above will work on any of CPM2.0, CPM2.22, CPM2.23 and the Xpress CPM. The method below works on CPM2.22, although the G command given below will work on all of them and the function key addresses can be obtained from there.

The alternative method is to hardcode the key definitions into the CPM system, so that the key definitions are defined when you start the system using that system disk. To do this you need to use DDT80. Type DDT80 SYSGEN.COM.

This loads the program SYSGEN.COM into memory. DDT will let you run this program, stopping it at any stage. Under SVI/CPM enter the DDT command

G100,1a4

and follow the usual SYSGEN prompts, which loads the CPM system into memory. Now you can change the function key definitions which are stored at locations 1FF5 to 2094. To do this you use the DDT command S1FF5 and enter the ascii codes which you want for your keys. This process ends with a period.

Having changed the definitions you now continue the SYSGEN process, by typing G and following the prompts as usual. These definitions are now part of the CPM system.

Whichever of the 2 methods you use (COM file or alter CPM with DDT), you will have a more user friendly setup with the function keys, as they now include the command, and a description of the command.

By the way, you can alter anything in CPM you like with the DDT method. For instance the 2 line message you see when you switch the system on will be at locations 28FC to 2942. You could change these bytes to anything you wish (maybe change 291e/291f to spaces, ascii 20, which will have the effect of changing the message to a one line message). There are many other messages you may wish to change in there as well.

In MSX, the equivalent addresses to the above are,

| | | |
|----------|--------|----------|
| G100,1a4 | -----> | G100,1D4 |
| 28Fc | -----> | 2EE4 |
| 2942 | -----> | 2FC1 |

| | | |
|------|--------|------|
| 1ff5 | -----> | 2193 |
| 2094 | -----> | 223A |

Under the BASIC operating system you can poke the definitions into the relevant locations. These are f87f --> f87f+159 for SVI and f87f --> f87f+159 under MSX-BASIC.

As an extra to the discussion about function keys, you can have key definitions longer than 16 characters. Just don't put a null character in and BASIC thinks that the definition doesn't end. you can get up to 40 character long keys this way, but no longer because 40 characters is the maximum keyboard buffer length. For example ...

```
A$ = "THIS IS A VERY VERY VERY VERY LONG KEY"
For I=1 to Len(A$)
  Poke &hfafe+i-1,asc(mid$(a$,i,1))
Next
poke &hfafe+len(a$),0
```

The tradeoff here is that if you define a key (say key 1) like this, then the next key would have the definition "RY VERY VERY LONG" which most likely is not very useful, so you need to avoid any urge to use the next function key.

(2) Computer Forum

Computer Forum is dedicated to SVI and MSX computers. At \$44 Aust a year this is value for money in a big way. The October issue will be my last column for Bits & Bytes, as I believe that Computer Forum, being a dedicated magazine with world-wide distribution can give far more support to SVI/MSX owners and I intend submitting articles to there from now on.

The address for Computer Forum is

```
Computer Forum
P.O Box 71
Palmyra
W.A 6157
Australia
```

(3) Rapid Help Systems

If you use 40 column basic heres a hint for those of you writing programs which use some form of help on screen. You can avoid needing to store the help information in your program by using a useful VDP feature, the ability to switch the currently seen area of vram to any 1024 byte boundary in vram.

A 1024 byte boundary is an area of vram with starting address a multiple of 1024. Thus 0,1024,2048,3072,4096 ... are the 1024 byte boundaries. The usual area seen on screen uses the

bytes starting at 0.

Under MSX the process is fairly simple. Vpoke the text you want to see into vram starting at 4096 (don't go any lower as this would eat into the character table) in blocks of 960 bytes (as 960 bytes are the maximum on each 40 column screen). You can fit 11 screens in, starting from 4096, then to 4096+1024, 4096+2048 ...

To look at a particular screen, say the one starting at 8196, use the BASE function. -Your program would have BASE(0)=8192. You use BASE(0) as this identifies where the current screen 0 text is in vram. To go back to your usual screen use BASE(0)=0.

The following program demonstrates this under MSX. All that happens is that a particular character is put in vram for each byte boundary. Then the BASE function is used to let you see each block. Note how rapid the process is (although the initial set up process of getting the bytes into vram is very slow, but need not be repeated unless you switch screen modes at some stage. Thus if you stop the program, you do not need to go back through the set up process again).

```
10 '  
20 ' set up the Vram stuff  
30 '  
40 WIDTH40 : CLS:LOCATE5,5:PRINT"Loading VRAM"  
50 DEFINT J,I,S,K  
60 K=ASC("A")-1  
70 FOR I=0 TO 10  
80   K=K+1  
90   FOR J=I*1024+4096 TO I*1024+5055  
100     VPOKE J,K  
110     LOCATE5,8:PRINT"Filling &h";HEX$(J);  
120   NEXT  
130 NEXT  
140 '  
150 ' demonstrate rapid screen changes  
160 '  
170 ONSTOPGOSUB240:STOP ON  
180 LOCATE 0,20:PRINT"Enter Screen To Look At (1 - 11)";  
190 LOCATE0,21:PRINT"Press any Key To return To This Screen";  
200 LOCATE0,22:PRINT"Press CNTL/STOP to finish program";  
210 LOCATE35,20:INPUT SC:IF SC<1 OR SC>11 THEN180  
220 SC=3072+SC*1024  
230 BASE(0)=SC:A$=INPUT$(1):BASE(0)=0:GOTO180  
240 LOCATE20,23:END
```

Under SVI/BASIC the BASE function is not available. So we need a little extra code. This is just replacing the BASE function with direct writing to the VDP output ports. Simply change 220 to read SC = SC + 3 and change 230 to read

```

230 OUT&HB1,SC:OUT&HB1,&HB2:A$=INPUT$(1):
    OUT&HB1,0:OUT&HB1,&HB2:GOTO180

```

What is being done in line 230 is that we are accessing the VDP register which controls the area of vram currently displayed. This is register 2. To access any VDP register the procedure is simple, send the data (SC) down to the VDP register port (81h), then send down the register number, but OR it with 80h first. It is a requirement of the VDP that the number sent to the register to define the displayed vram is divided by 1024. Thus to set 4096 as the start of the currently defined screen, send down $4096/1024 = 4$.

This will not work on the SVI 80 column screen because it does not use the vram, however it will work on the MSX Xpress 80 column screen in MSX-DOS, BASIC or CFM as the screen is displayed through the VDP. Since the Xpress appears to have a degree of print spooling (under CFM) by using the VDP as a large print buffer and printing only during interrupts, conflict could arise when changing the displayed screen and printing.

WANTED

Single - or Dual Disk-drive expander unit for SV 328 Mk II.

F. Sutherland
 33 Kukorwald St
 Wallacetown
 Southland

Phone: 358034

+++++



"We may as well go home. It's obvious that this meeting isn't going to settle anything."

PUBLIC DOMAIN SOFTWARE

There are now 5 more Public Domain Computer Program tapes available. The contents of these five are listed below. A list of the first 9 tapes was given in the December 1983 issue, page 11 of this magazine.

- TAPE 10 Includes 5 Top games from Aust. Users group.
Volley Ball, Save your soul, Test pattern/Clock,
Diamond Miner, Music (Annie), Wildcat oil
strike, Sub Hunt, 80 column picture of a lady,
Sound (Helicopter), Tattsлото, Temp converter,
Play the piano, Colour and Paint, Palindrome
Numbers, 12 Days of Xmas, Sound (Telephone).
- TAPE 11 5 Games and 14 Utility programs.
Collision, Hangman, Shoot Game, Mirror Game,
Pacman, Sprite maker, Draw House, Happy Birth-
day, Sprite sizes, 3D plot, Screen 2 scrolling,
Mc display, Trace Variables, Rat Trap Security
Font Screen print, Font enlargement, Screen swap
Redefine Errors, CMD Repeat and Until.
- TAPE 12 20 Programs.
Put memory into data lines, Machine Code Header
Reader, Rotate name table, Line cmd variations
Screen 1 appears instantly, Border rotates, 32
Sprites move, Astro Blaster, Metronome timer,
Maths tester, Blasting Duel, Mastermind,
Aleyin Invasion, Garden Record keeper, Program
directory file, Colour screen/border/sprite,
Sprite, Snoopy picture, 4-row, Recipe file.
- TAPE 13 7 Games, 3 Graphics programs and utilities.
Numble Marble puzzle, Bathurst car race, Save
the penguin eggs, Evasion, IBM logo, Spiral
graphics, Point graphics, 9 Hole golf, Airfox
adventure action game, Doctor analyst, 2 Word
sort progs, anagram, Bank switching - 29k more
ram!, Naval combat board game, Music Tutorial.
- TAPE 14 Puzzles, Games and Utilities.
Printer Screen DUMP II, Banner for printers,
Sort by count, Hidden words puzzle, Calc Spread
Sheet 80 columns, Life expectancy, Grand Prix
racing game, Russian Roulette, Poker Machine,
Horse Racing, Using words, Address label,
Quest adventure game, Poetry composer, Toad
crossing the road game.

Software can be ordered from; SV PD Software,
P.O.Box 26050,
Newlands, Wellington
New Zealand.

Costs: Each tape is \$3 each with a handling
charge of \$2.50 for each order made.
Cheques to be made payable to G.Clark.

Note: Programs from this list are broadcast on
Access Radio on the 1st Saturday of every
month at 9.00am on station 2YB 783 kHz.